6 Evaluation

We evaluate how the implemented datapath turns parallel packet execution into completed messages. Three questions guide the experiments. Can the transmitter use several paths when an application exposes only a few QPs? How much disorder can the receiver absorb before its finite window becomes a constraint? What does ordered completion cost when the DMA lanes finish at different times? The results show both the benefit and the limits of the current implementation: packet spraying raises single-QP goodput by up to 2.48× in the tested workloads, but its advantage depends on segmentation and the number of active QPs. At the receiver, window pressure and global retirement introduce two distinct constraints that network bandwidth alone does not remove.

6.1 Experimental Method

We connect a Verilator simulation of the TX RTL, an ns-3 network, and a Verilator simulation of the RX RTL through Unix sockets. TX exports each frame with the simulation timestamp at which its final beat leaves the packet scheduler. ns-3 injects that frame at the supplied timestamp, models link serialization, queuing, and propagation, and streams arrival events to RX. Timestamped advance messages coordinate the three processes. The simulator therefore preserves the timing of packet production while allowing the network to change arrival order. Table 1 lists the configuration used in this batch.

Table 1: RTL and network simulation configuration. Link rates and the RTL clock are model parameters.

Component	Configuration
Toolchain	BSC 2023.01, Verilator 5.048, ns-3 3.48, cocotb 2.1.0.dev0
RTL	2 ns cycle; 4 QPs and 4 receive lanes; messages up to 16 KiB
Receive state	16-packet window per QP; 16 payload slots per lane; 4 Ordered Queue entries per QP
Network	Two nodes, four independent 100 Gb/s point-to-point links; 100-packet queue per link
Path delay	200 ns on all paths for throughput and policy comparisons; controlled skew for receive-pressure experiments
DMA model	TX completions returned in reverse batches of up to 8 requests; RX completion acknowledged after payload collection
Packet profile	Ethernet/IPv4/UDP/BTH/RETH; ICRC and FCS not modeled

The prototype exercises request processing, packetization, path selection, receive-window handling, and modeled DMA placement. Ordered Queue and retirement experiments additionally isolate their RTL modules so that completion delays can be controlled. This setup does not include a GPU, physical PCIe, a switched congestion topology, or a network ACK-to-TX retransmission loop. Accordingly, we report datapath behavior and model-level timing; GPU submission overhead, GPU-visible completion latency, loss recovery, and target-FPGA frequency require separate measurements. The four paths also evaluate path selection independently of the proposed SRv6 encapsulation.

We use three timing boundaries. *TX latency* starts when the RTL accepts a request. Its end is either the first packet's start of frame (SOF) or the latest end of frame (EOF) among all packets belonging to that message. *TX effective payload rate* divides the batch's payload bits by the interval from the first accepted request to the last emitted frame. *Network goodput* instead uses the interval from the first frame injection to the last frame arrival. For an individual message, *network completion time* runs from its earliest frame injection to the arrival of all its frames. This last metric includes gaps between a message's packet injections,

but excludes request processing before its first injection and receiver processing after arrival.

All plotted points are deterministic runs with the specified workload. The throughput experiments use finite batches without a separate warm-up interval; we do not interpret their rates as steady-state NIC throughput. Likewise, the eight-message policy comparisons support paired means, not tail-latency estimates. We regenerate the plotted values from request, frame, and network event records and check them against the archived summaries.

Before using these timings, we check that the coordination mechanism does not change the reference workload. With advance windows of 100 ns, 1 μs, and 5 μs, the simulation takes 12, 3, and 2 advance rounds, respectively, yet produces identical per-frame injection times, paths, and arrival events. A free-clock variant and offline replay reproduce the same events. Across the five modes, all 20 per-path packet captures match their corresponding reference bytes and timestamps. Final simulator drain time varies with the advance window and is excluded from this comparison. The functional checks in Table 2 establish the scope of the accompanying RTL tests.

Table 2: Functional validation in the archived experiment batch. Short and long RX suites overlap and are reported separately.

Check	Exercised behavior	Result
TX regression	Request processing, DMA, packetization, scheduling	64 pass
RX short regression	Receive checks, placement, retirement, Ordered Flags	37 pass, 1 skip
RX long regression	Extended receive and completion tests	38 pass
Online reference	6 messages across 4 QPs and unequal-delay paths	11/11 frames; 0 drops
Timing consistency	Three window sizes, free-clock mode, offline replay	Exact event match
Reverse DMA completion	Completion order $3, 2, 1, 0$ across four QPs	Retire order $0, 1, 2, 3$
Ordered Queue capacity	1, 2, and 4 Flags; a fifth blocked Flag	Credits match; overflow detected

6.2 From an Accepted Request to Packet Output

We first isolate the transmitter because the network can exploit only the parallelism that TX supplies. Figure 1 separates the delay before the first packet starts from the time needed to emit the whole message. With a 4 KiB PMTU, a 64 B request starts its first packet after 102 ns and finishes after 118 ns. For 16 KiB, those times become 510 ns and 1,738 ns. The first-packet delay remains 510 ns at both 8 KiB and 16 KiB, while whole-message emission continues to grow. Starting a message and draining its packets are thus different costs in this pipeline.

Segmentation makes this distinction particularly important. In the 4 KiB case, the current packet format produces payload fragments of 4,068 B and 28 B. The small tail can finish on another lane before the large first packet. Its EOF occurs 380 ns after request acceptance, whereas the complete message leaves TX at 942 ns. Using the highest packet index as the completion marker would underestimate emission time by 562 ns. We use the latest EOF throughout.

For a fixed 2 KiB message, reducing the PMTU from 4,096 B to 256 B changes the packet count from one to nine and reduces whole-message emission from 522 ns to 284 ns in this model. Smaller packets permit

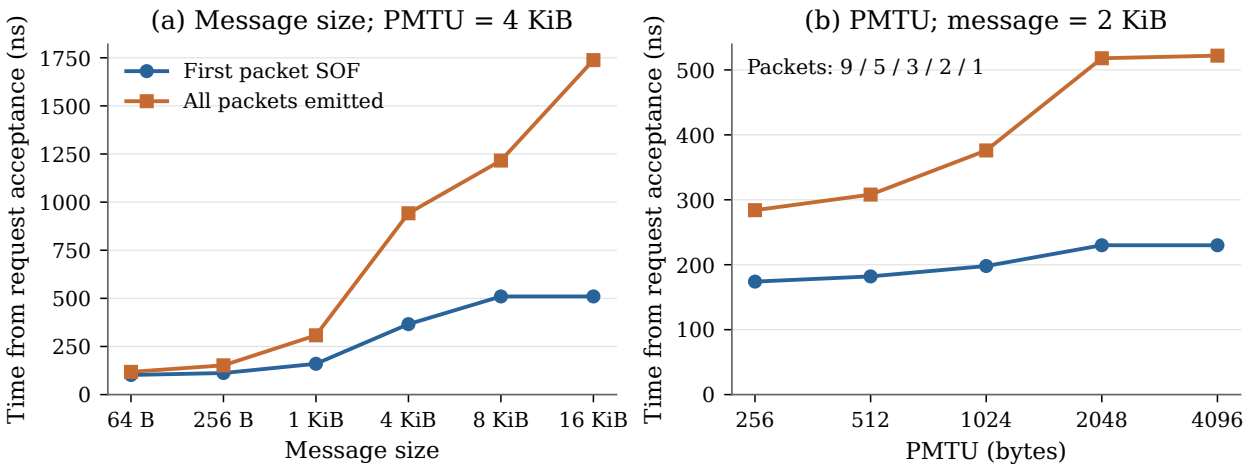


Fig. 1: Isolated TX latency from request acceptance. "All packets emitted" uses the latest EOF across the message, including when a small tail packet finishes before an earlier packet. PMTU points in (b) use one 2 KiB message.

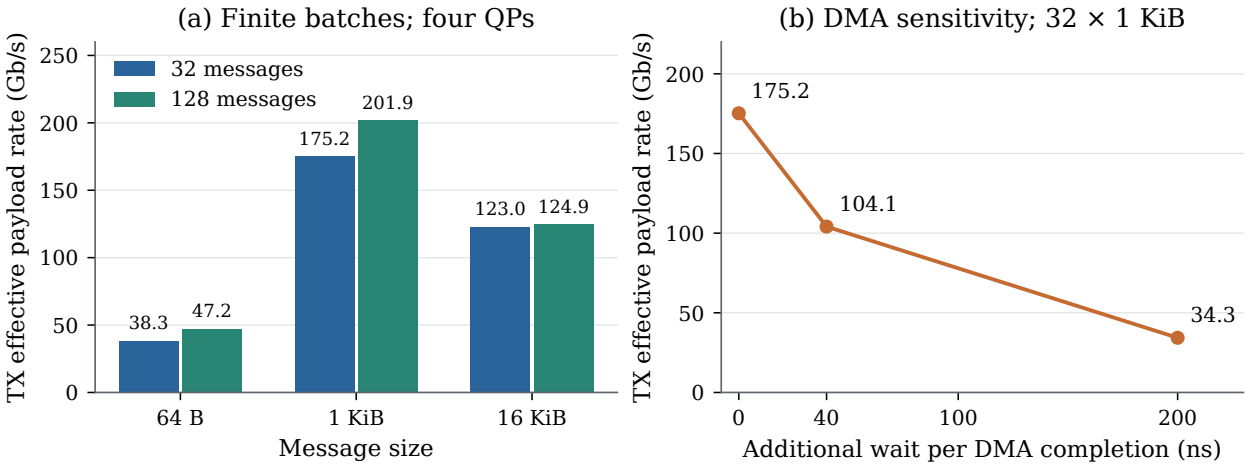


Fig. 2: TX payload rate over finite batches on four QPs. (a) Increasing the batch length reduces the contribution of startup and drain. (b) Additional waiting before each modeled DMA completion reduces the rate of a 32-message, 1 KiB workload. All paths have 200 ns propagation delay.

more work to proceed across the output lanes, at the cost of additional packet processing and headers. This experiment measures the TX side of that tradeoff; it does not establish an optimal PMTU for a network workload.

Figure 2(a) then keeps the pipeline busy with batches of 32 and 128 messages. At 1 KiB, TX effective payload rate rises from 175.2 to 201.9 Gb/s as the batch grows; the corresponding network goodput is 177.9 and 202.8 Gb/s. The two rates use different start boundaries and should not be subtracted as a network overhead. For 64 B messages, TX reaches 47.2 Gb/s with 128 messages. For 16 KiB messages, it reaches 124.9 Gb/s, close to the 123.0 Gb/s of the shorter batch. Larger messages therefore do not yield the highest rate in the current pipeline and DMA model. Assigning that limit to a particular RTL resource would require stall attribution beyond these traces.

The controlled DMA experiment exposes one concrete dependency. Adding 20 or 100 cycles of wait before each DMA completion lowers the 1 KiB, 32-message TX rate from 175.2 Gb/s to 104.1 or 34.3 Gb/s (Figure 2(b)). Four egress paths cannot compensate for data that has not yet returned to the packetization pipeline. This

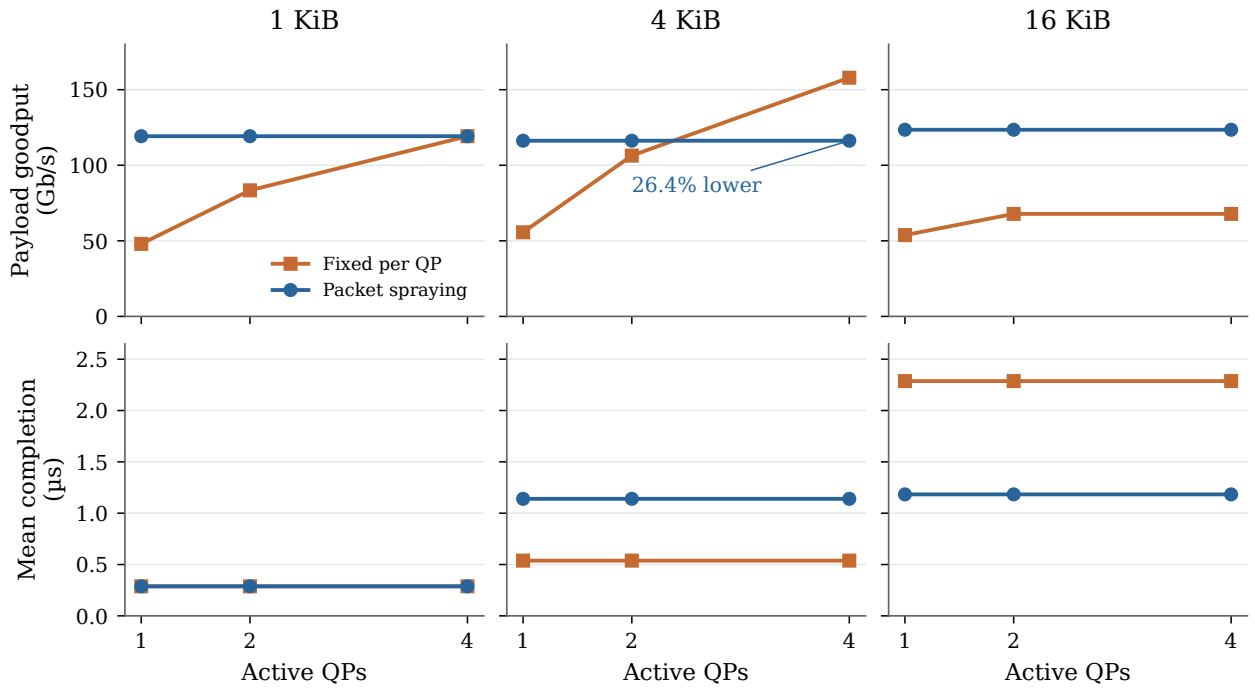


Fig. 3: Packet spraying versus a fixed path per QP. Every point represents eight messages. The upper row reports batch network goodput; the lower row reports mean time from a message's first injection to its last arrival. Spraying benefits a single QP, but four QPs with 4 KiB messages favor fixed paths in this implementation.

motivates examining path selection under the same DMA model, rather than attributing every throughput difference to the network.

6.3 When Packet Spraying Helps

We compare packet spraying with a fixed-path policy that maps each QP to one path using the low two bits of its QPN. Spraying uses the implemented round-robin selection among ready paths. Each pair has the same offered sequence of eight messages, message size, QP assignment, PMTU of 4 KiB, and four identical 200 ns paths. Policy-dependent backpressure may change the actual acceptance and injection times. The fixed policy is an implementation-level baseline for path selection, not a comparison with a commercial RNIC.

With one active QP, a fixed assignment leaves three paths unused. Spraying raises 1 KiB goodput from 48.0 to 119.2 Gb/s, a 2.48× increase, and 16 KiB goodput from 53.8 to 123.5 Gb/s, a 2.30× increase (Figure 3). For 16 KiB messages, mean network completion also falls from 2.287 to 1.183 μs. In contrast, each 1 KiB message occupies a single packet, whose 287.68 ns injection-to-arrival latency is unchanged. The 1 KiB gain comes from distributing different messages of the same QP across paths, rather than shortening an individual packet's transit time.

As more QPs become active, the fixed policy can use more links. At four QPs, the 1 KiB policies converge at 119.2 Gb/s. The 4 KiB case goes further: fixed paths reach 157.9 Gb/s, while spraying reaches only 116.2 Gb/s, or 26.4% less. Mean network completion under spraying is 1.140 μs, compared with 0.538 μs under fixed paths.

The frame trace helps explain why packet spreading need not mean byte balance. Each 4 KiB message contains one 4,068 B fragment and a 28 B tail. With four QPs, spraying assigns 12,204, 8,192, 8,164, and

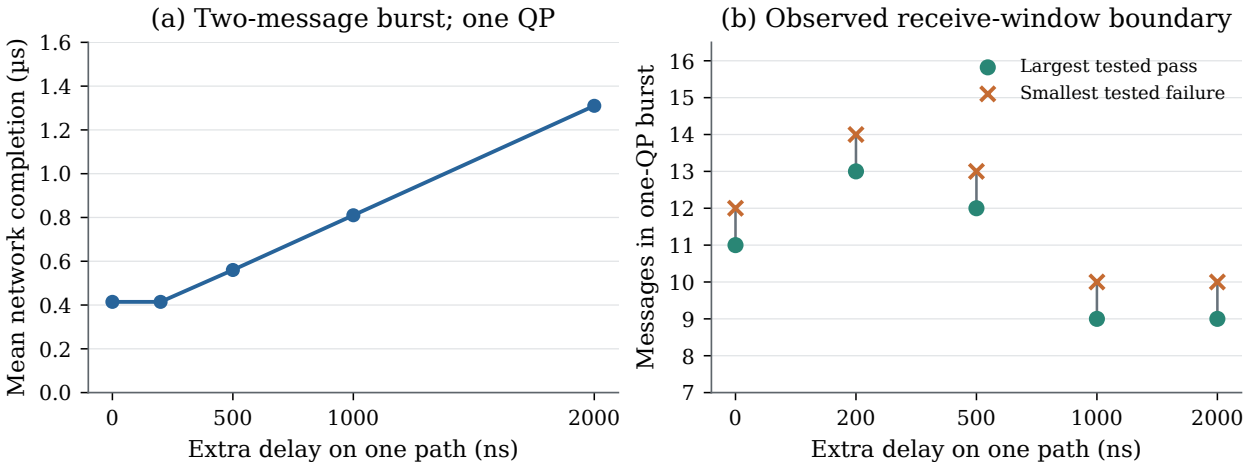


Fig. 4: Receive behavior with three 200 ns paths and one slower path. Messages are 1 KiB with a 1 KiB PMTU. (a) Mean network completion for a two-message burst. (b) Largest tested passing and smallest tested failing one-QP bursts. Failure is an RX out-of-window error; all frames still arrive through the network. Marker pairs describe tested cases, not measured buffer occupancy or a general capacity bound.

4,208 payload bytes to the four paths; fixed assignment gives each path 8,192 B. Small tails also enter the network before their large first fragments, extending the interval from first injection to whole-message arrival. Thus the spraying result combines uneven byte distribution with a different packet-emission schedule. The experiment does not isolate their individual contributions, but it identifies a useful design constraint: a policy that balances packet opportunities can still distribute bytes poorly near a segmentation boundary. The measured single-QP benefit is real, but it is not a uniform improvement over fixed paths.

6.4 Receive Progress Under Path Skew

The previous experiment uses symmetric paths. We next ask how far the receiver can sustain progress when their delays diverge. A 1 KiB message with a 1 KiB PMTU produces two packets, carrying 996 B and 28 B. We keep three paths at 200 ns and add up to 2,000 ns to the fourth. A short two-message burst on one QP separates completion latency from the pressure caused by a longer burst.

Mean network completion stays at 414.44 ns when the extra delay increases from zero to 200 ns, then rises to 560.08, 810.08, and 1,310.08 ns at 500, 1,000, and 2,000 ns of extra delay (Figure 4(a)). In the initial range, the delayed small fragment is not yet the message's last arrival. Once it becomes the last arrival, additional path delay extends completion. All 21 short tests across seven delay profiles and one, two, or four QPs complete successfully. The four-QP tests use four messages rather than two and are not used as an equal-load scaling comparison.

Longer bursts reveal a different limit. With equal path delays, 11 messages pass and 12 encounter `RX_OUT_OF_WINDOW`. With an extra 1,000 or 2,000 ns on one path, nine pass and ten fail (Figure 4(b)). These are receiver-window failures rather than network loss: every frame is delivered in the recorded failing runs. Without a closed retransmission loop, the batch cannot recover from those rejections.

Equal propagation delay does not imply in-order input to RX. The small tail may leave TX before its first fragment, and packets assigned to different lanes have different serialization histories. In the equal-delay 12-message run, all 24 frames arrive, but PSNs 21 and 20 encounter the receive-window boundary. Progress depends on which missing packet releases the window, not only on the number of bytes in flight. The observed threshold is consequently not monotonic in path skew: an extra 200 ns permits 13 messages before the first

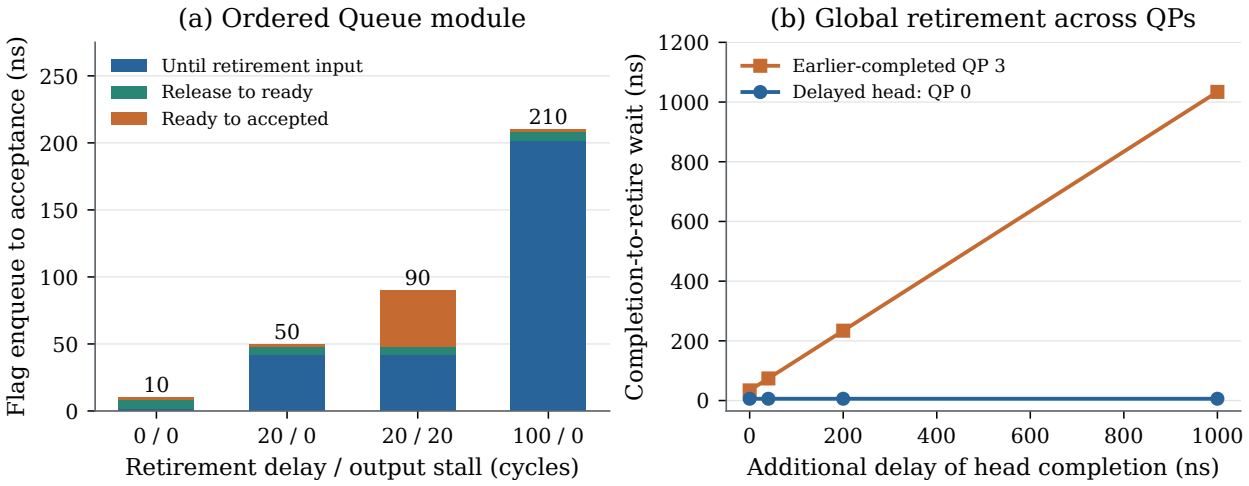


Fig. 5: Controlled RTL tests of completion ordering. (a) Flag release with delayed retirement input and output backpressure; one cycle is 2 ns. (b) Four different QPs complete in order $3, 2, 1, 0$ and retire in order $0, 1, 2, 3$. Delaying QP 0 propagates into the wait of QP 3. Times are testbench handshake observations, including its drive and drain cadence.

tested failure at 14. This is a property of the tested schedule, not evidence that adding delay generally increases capacity. The results motivate matching the offered burst and sender control to receive-window progress; a wider window alone would need its own capacity sweep to establish the benefit.

6.5 Ordered Completion and Its Blocking Cost

Arrival of all payload packets does not by itself authorize an Ordered Flag. We therefore test the release and retirement stages separately. In the first test, the Ordered Queue receives a Flag and then waits for the testbench to supply retirement evidence reaching its barrier. The event recorded as payload retirement is an injected module input in this experiment; it is not a measured GPU-memory DMA completion.

Without output backpressure, retirement-input delays of zero, 20, and 100 cycles produce observed Flag enqueue-to-acceptance times of 10, 50, and 210 ns (Figure 5(a)). In these traces the request becomes ready 6 ns after the retirement input is accepted. Adding 20 cycles of output stall to the 20-cycle retirement-delay case moves Flag acceptance from 50 to 90 ns. The queue thus responds to both the barrier condition and the availability of the downstream write path.

The event order also separates resource release from write completion. In the 100-cycle case, the Flag is enqueued at relative time zero, retirement evidence is accepted at 202 ns, the Flag request becomes ready at 208 ns and is accepted at 210 ns, a credit is dequeued at 212 ns, and a modeled Flag completion response is accepted at 214 ns. The two-nanosecond observation gaps include sequential testbench operations and are not intrinsic interface latency claims. What matters here is that credit return follows request acceptance without waiting for the Flag completion response. Separate capacity tests return one, two, and four credits for the corresponding released Flags; a fifth Flag presented to a blocked four-entry queue triggers the configured overflow protection.

The second test exposes the cost of a single global retirement order. We place one request from each of four QPs on different DMA lanes, then complete them in reverse sequence. Retirement remains $0, 1, 2, 3$ for every tested head delay. However, QP 3, whose DMA finishes first, waits 34 ns to retire in the baseline and 1,034 ns when QP 0's completion is delayed by an additional 1,000 ns (Figure 5(b)). The head request's own observed

completion-to-retire interval stays at 6 ns. The added delay is therefore transferred to requests belonging to other QPs even though their DMAs have already completed. This is the cross-QP head-of-line blocking accepted by the current design. Per-QP retirement is a possible way to reduce it, but its extra state and arbitration are not evaluated here.

6.6 State Footprint and Remaining Scope

The implementation makes the cost of ordering explicit in its state structures. Table 3 groups the archived RTL capacity inventory. The listed structures contain 2,586,432 logical bits, or 315.7 KiB. RX frame storage accounts for 260 KiB of that total, whereas the four per-QP Ordered Queues together hold 416 B of entry data. Payload buffering, rather than Flag metadata, dominates this inventory. This distinction matters because adding space for waiting Flags does not address receive-packet window pressure or global retirement blocking.

Table 3: Logical storage capacity of the listed RTL structures. Counts describe declared entry data, not placed FPGA memory or a complete area estimate. KiB denotes 1,024 bytes.

Structure	Logical bits	KiB
RX payload frame storage	2,129,920	260.00
TX per-channel packet payload storage	262,144	32.00
TX DMA completion reorder storage	158,848	19.39
RX pending-packet table	13,888	1.70
Per-QP Ordered Queues (16 entries total)	3,328	0.41
Other listed tables, counters, and bitmaps	18,304	2.23
Total listed capacity	2,586,432	315.73

The inventory excludes unlisted control and FIFO overhead, and synthesis may change memory packing substantially. The available generic-cell synthesis report covers only the TX scheduler; it cannot provide a full-NIC LUT count, BRAM utilization, or achievable frequency. We therefore leave those quantities unreported. The present evidence establishes functional datapath execution, conditional multipath gains, and concrete receive and completion limits. Extending the evaluation to a complete system requires a closed-loop loss and recovery experiment, GPU submission and visibility measurements, and target-device synthesis. Those measurements are also needed before attributing end-to-end application speedup or resource savings to OpenRDMA.